

Passo a Passo do Sistema SENAI — Controle de Acesso Escolar

Documentação do código atual: como o sistema funciona hoje, do banco até cada perfil de usuário.

Visão geral

O SENAI — Controle de Acesso é um sistema Laravel para escolas. Três perfis principais:

Perfil	Papel no banco	O que faz
AQV	aqv	Cadastra cursos, alunos, equipe e solicitações de saída
Professor	professor	Recebe notificações quando um aluno vai sair
Porteiro	portaria	Libera a saída na portaria

Passo 1 — Configuração do banco no .env

O Laravel precisa estar ligado ao MySQL.

Arquivo: .env

DB_CONNECTION=mysql

DB_HOST=127.0.0.1

DB_PORT=3306

DB_DATABASE=safe_senai

DB_USERNAME=root

DB_PASSWORD=

Configuração	Função
DB_CONNECTION=mysql	Usa MySQL
DB_HOST	Servidor do banco
DB_PORT	Porta (3306)
DB_DATABASE	Nome do banco criado no Laragon
DB_USERNAME / DB_PASSWORD	Credenciais

Depois de configurar:

```
php artisan migrate
```

```
php artisan db:seed
```

Isso cria as tabelas e o usuário AQV padrão (izete@safe.com / izete123).

Passo 2 — Estrutura do banco (migrations)

As tabelas são criadas pelas migrations em database/migrations/.

Tabela users

Usuários do sistema (login).

Campo	Função
name	Nome
email	E-mail (login)
password	Senha (criptografada)
role	aqv, professor ou portaria
cpf, telefone, matricula	Dados da equipe
curso	Curso do professor (texto)
turno, setor	Turno e setor
ativo	Se pode fazer login

Tabela cursos

Campo	Função
nome	Nome do curso
tipo	kai, tecnico ou fic
vagas	Limite de alunos
cadastrado_por	AQV que cadastrou

Regra: combinação nome + tipo é única.

Tabela curso_professor

Liga curso e professor (até 5 por curso).

Campo	Função
curso_id	Curso
user_id	Professor

Tabela alunos

Campo	Função
nome_completo	Nome do aluno
cpf	CPF (11 dígitos, único)
idade	Idade
responsavel_nome	Responsável
responsavel_telefone	Telefone
curso_id	Curso (FK)
cadastrado_por	AQV que cadastrou

Tabela cards_saida

Solicitações de saída.

Campo	Função
aluno_id	Aluno
diretor_id	AQV que criou
horario_saida	Horário
responsavel_autorizou	Quem autorizou
qtd_faltas	Quantidade de faltas
aulas_falta	Aulas (JSON)
status	pendente ou liberado
liberado_em, liberado_por	Liberação na portaria

Tabela notificacoes

Campo	Função
user_id	Professor ou porteiro
card_saida_id	Solicitação
tipo	professor ou portaria
titulo, mensagem	Texto do aviso
lida_em	Quando foi lida

Passo 3 — Login e entrada no sistema

Tela de login

Arquivo: resources/views/welcome.blade.php

Layout: resources/views/layouts/auth-guest.blade.php

- Aba Entrar: e-mail e senha
- Aba Primeiro acesso: orientações (conta criada pela AQV)
- Visual SENAI: vermelho #e30613, fundo escuro, logo

Envio do login

```
<form method="POST" action="{{ route('login') }}">

    @csrf

    <input name="email" ...>

    <input name="password" ...>

</form>
```

Controller de autenticação

Arquivo: app/Http/Controllers/Auth/AuthenticatedSessionController.php

1. Valida e-mail e senha
2. Bloqueia se ativo = false
3. Cria sessão
4. Redireciona para /home

Redirecionamento por perfil

Arquivo: app/Http/Controllers/HomeController.php

```
return match ($user->role) {  
    'aqv' => redirect()->route('aqv.dashboard'),  
    'professor' => redirect()->route('professor.dashboard'),  
    'portaria' => redirect()->route('portaria.dashboard'),  
    default => redirect()->route('login')->with('error', ...),  
};
```

Cada perfil vai ao painel correto.

Passo 4 — Controle de acesso (middleware)

Arquivo: app/Http/Middleware/CheckRole.php

Registro: bootstrap/app.php → alias role

```
if (! in_array($role, $roles, true)) {  
    return redirect()->route('home')->with('error', 'Acesso negado...');  
}
```

Exemplo nas rotas:

```
Route::middleware('role:aqv')->group(function () {  
    // só AQV  
});
```

Sem o papel certo, o usuário volta para /home com mensagem de erro.

Passo 5 — Layout interno (painel logado)

Arquivo: resources/views/layouts/safe.blade.php

- Cabeçalho com logo SENAI (<x-senai-logo />)
- Menu conforme o perfil (AQV: Início, Cursos, Alunos, Equipe, Solicitações)
- Cards, tabelas, alertas, botões
- Máscaras de CPF e telefone: public/js/safe-masks.js

Logo: app/Support/SenaiBrand.php busca imagem em public/images/ (ex.: senai-logo.png).

Passo 6 — Módulo de cursos (AQV)

Listagem

Rota: GET /cursos → CursoController@index

View: resources/views/cursos/index.blade.php

Lista cursos com tipo, vagas, alunos (6 / 40), professores e ações Editar / Excluir.

Formulário

View: resources/views/cursos/_form.blade.php

```
<form method="POST" action="{{ route('cursos.store') }}">

    @csrf

    <input name="nome" required>

    <select name="tipo"> KAI / TECNICO / FIC </select>

    <input name="vagas" type="number" min="1" max="500">

    <input type="checkbox" name="professor_ids[]" value="...">

</form>
```


Rota de cadastro

```
Route::post('/cursos', [CursoController::class, 'store'])->name('cursos.store');
```

Salvamento — CursoController@store

1. Validação

```
$validated = $request->validate([  
    'nome' => 'required|unique:cursos,nome + tipo',  
    'tipo' => 'required|in:kai,tecnico,fic',  
    'vagas' => 'required|integer|min:1|max:500',  
    'professor_ids' => 'nullable|array|max:5',  
]);
```

2. Gravação

```
$curso = Curso::create([  
    'nome' => trim($validated['nome']),  
    'tipo' => $validated['tipo'],  
    'vagas' => $validated['vagas'],  
    'cadastrado_por' => auth()->id(),  
]);  
  
$curso->professores()->sync($validated['professor_ids'] ?? []);
```

3. Model

Arquivo: app/Models/Curso.php

```
protected $fillable = ['nome', 'tipo', 'vagas', 'cadastrado_por'];
```

Relações: professores(), alunos(), cadastradoPor().

Exclusão: só se não houver alunos no curso.

Passo 7 — Módulo de alunos (AQV)

Formulário

View: resources/views/alunos/_form.blade.php

- Nome, CPF (máscara), idade
- Responsável e telefone
- Curso em <select> só com cursos cadastrados e vagas disponíveis

Rota

```
Route::post('/alunos', [AlunoController::class, 'store'])->name('alunos.store');
```

Salvamento — AlunoController@store

1. Normaliza CPF (só números)

2. Valida

```
'curso_id' => 'required|exists:cursos,id',
```

```
'cpf' => 'required|size:11|unique:alunos,cpf',
```

3. Verifica vagas

```
if ($curso->alunos()->count() >= $curso->vagas) {  
    return back()->with('error', 'Limite de vagas atingido.');
```

```
}
```

4. Salva

```
Aluno::create([...$validated, 'cadastrado_por' => auth()->id()]);
```

Model: app/Models/Aluno.php — relação curso() → Curso.

Passo 8 — Módulo de equipe (AQV)

Rota base: /usuarios

Controller: UsuarioController

Cadastra professor e porteiro:

- Nome, e-mail, senha, CPF, telefone
- Professor: curso opcional, matrícula, turno
- Porteiro: setor

```
User::create([
    ...$validated,
    'ativo' => true,
    'cadastrado_por' => auth()->id(),
    'email_verified_at' => now(),
]);
```

Desativar: PATCH /usuarios/{usuario}/ativo — usuário inativo não entra.

User::cargoLabel() exibe AQV, Professor ou Porteiro na interface.

Passo 9 — Solicitações de saída (AQV)

Formulário

View: resources/views/solicitacoes/create.blade.php

```
<form method="POST" action="{{ route('solicitacoes.store') }}">

    @csrf

    <select name="aluno_id">...</select>

    <input name="horario_saida" type="time">

    <input name="responsavel_autorizou">

    <input type="checkbox" name="aulas_falta[]" value="1..5">

</form>
```

Rota

```
Route::post('/solicitacoes', [CardSaidaController::class, 'store']);
```

Fluxo — CardSaidaController@store

1. Valida aluno, horário, responsável e aulas
2. Chama CardSaidaService::criar()
3. Redireciona com: *“Professor e porteiro foram notificados.”*

Serviço — CardSaidaService

Arquivo: app/Services/CardSaidaService.php

```
$card = CardSaida::create([
    'aluno_id' => $aluno->id,
    'diretor_id' => $aqvId,
    'horario_saida' => ...,
    'status' => 'pendente',
]);

// Notifica professores do curso
foreach ($aluno->curso->professores as $professor) {
    Notificacao::create([...]);
}

// Notifica todos os porteiros
foreach (User::where('role', 'portaria')->get() as $porteiro) {
    Notificacao::create([...]);
}

Tudo em transação (DB::transaction).
```

Passo 10 — Painel do professor

Rota: GET /professor

Controller: ProfessorDashboardController

View: resources/views/professor/dashboard.blade.php

- Lista notificações de saída
- Marcar como lida: POST /notificacoes/{notificacao}/lida

Passo 11 — Painel do porteiro

Rota: GET /portaria

Controller: PortariaDashboardController

View: resources/views/portaria/dashboard.blade.php

- Solicitações pendentes
- Botão Liberar saída

Liberação

```
Route::post('/solicitacoes/{card}/liberar', [CardSaidaController::class, 'liberar']);

$card->update([

    'status' => 'liberado',

    'liberado_em' => now(),

    'liberado_por' => auth()->id(),

]);
```

Só libera se status === pendente.

Passo 12 — Rotas completas (resumo)

Arquivo: routes/web.php

Perfil	Principais rotas
Público	/, /login, recuperação de senha
Todos logados	/home, /profile, logout
AQV	/aqv, /cursos/*, /alunos/*, /usuarios/*, /solicitacoes/*
Professor	/professor, /notificacoes/{id}/lida
Porteiro	/portaria, /solicitacoes/{card}/liberar

Autenticação extra: routes/auth.php (Breeze).

Passo 13 — Models e relacionamentos

User

└─ cursosEnsino() —► Curso (pivot curso_professor)

└─ cadastradoPor() —► User

Curso

└─ professores() —► User

└─ alunos() —► Aluno

└─ cadastradoPor() —► User

Aluno

└─ curso() → Curso

└─ solicitacoesSaida() → CardSaida

CardSaida

└─ aluno() → Aluno

└─ diretor() → User (AQV)

└─ liberadoPor() → User

└─ notificacoes() → Notificacao

Notificacao

└─ usuario() → User

└─ cardSaida() → CardSaida

Passo 14 — Fluxo completo (exemplo)

1. AQV faz login → /aqv
 2. Cadastra curso 3DEVT (TECNICO, 40 vagas, professor Bruno)
 3. Cadastra aluno no curso
 4. Cria solicitação de saída (horário + aulas com falta)
 5. Sistema grava em cards_saida e cria notificacoes
 6. Professor vê aviso em /professor
 7. Porteiro libera em /portaria
 8. Status vira liberado
-

Arquivos principais

Área	Caminho
Rotas	routes/web.php, routes/auth.php
Middleware	app/Http/Middleware/CheckRole.php
Controllers	app/Http/Controllers/*
Serviço de saída	app/Services/CardSaidaService.php
Models	app/Models/
Migrations	database/migrations/
Seeders	database/seeders/AqvSeeder.php
Layout painel	resources/views/layouts/safe.blade.php
Login	resources/views/welcome.blade.php
Logo	app/Support/SenaiBrand.php, resources/views/components/senai-logo.blade.php

Credencial padrão (seeder)

Campo	Valor
E-mail	izete@safe.com
Senha	izete123
Perfil	AQV

Funcionalidades do sistema atual

1. Login com perfis AQV, professor e porteiro
2. Cursos KAI / TECNICO / FIC com vagas e professores
3. Alunos vinculados a curso com limite de vagas
4. Equipe (professor e porteiro) com ativação/desativação
5. Solicitações de saída com notificação automática
6. **Liberação na portaria com registro de quem liberou**
7. **Interface SENAI (vermelho/preto, logo, tabelas e formulários padronizados)**